Orbit Track — Dossier de lancement

Rendu 1 · v0.1.0 · novembre 2026

Version	Date	Auteurs	Changements
0.1.0	10/11/2026	S. Baibou, C. Chioukh, R. Baaboura	Création du dossier de lancement

> Les éléments notés **[à confirmer]** (API externe, fournisseur d'IA) seront fixés lors du premier sprint.

1. Équipe

#	Membre	N° étudiant	GitHub	Rôle principal	Feature majeure (responsable)
1	**Selma BAIBOU**	43001407	selmabayb	Back-end / communauté	**Participation, forum par événement et alertes** (rôles USER/ADMIN, modération, limite de débit, notifications in-app)
2	**Celia CHIOUKH**	43006515	celia050	Front-end / contenus	**Carte Leaflet, export .ics et articles quotidiens assistés par IA** (tâche planifiée, relecture, repli « Le savais-tu ? »)
3	**Ritej BAABOURA**	42007200	dontuseitiguess	Back-end / données	**Import & normalisation des événements** depuis les API astronomiques, cache Redis, mode dégradé, lieu conseillé

Chaque membre est responsable de sa feature de bout en bout (modèle de données, API, écrans, tests). Les tâches transverses (DevOps, authentification, revue de code) sont réparties par tickets et toute Pull Request est relue par un autre membre avant fusion.

2. Comptes développeurs utilisés

Service	Usage	Compte(s)
GitHub (organisation)	Dépôts, issues, GitHub Actions, GHCR (images Docker)	`selmabayb` (Selma), `celia050` (Celia, propriétaire du dépôt), `dontuseitiguess` (Ritej)
API astronomiques externes	Prévisions d'événements (éclipses, pluies d'étoiles filantes, passages, conjonctions…)	Compte projet, clé stockée en secret GitHub `[à confirmer]`
Fournisseur de LLM	Génération des brouillons d'articles	Compte projet `[à confirmer]`, clé en secret, quota plafonné
OpenStreetMap / tuiles	Fond de carte Leaflet (attribution affichée)	Sans compte (usage conforme à la politique de tuiles)
SonarCloud, Codecov	Qualité de code, couverture de tests (badges)	Liés à l'organisation GitHub

Aucune clé n'est versionnée : fichiers `.env` ignorés par Git, `.env.example` fourni, secrets injectés par GitHub Actions et Ansible Vault.

3. Le projet et son caractère innovant

Orbit Track aide les passionnés et les curieux à **ne rater aucun phénomène céleste et à l'observer ensemble**. L'application agrège des prévisions astronomiques issues d'API externes, recommande un site d'observation adapté, permet de s'inscrire à une sortie, d'échanger dans un forum dédié et d'ajouter l'événement à son agenda.

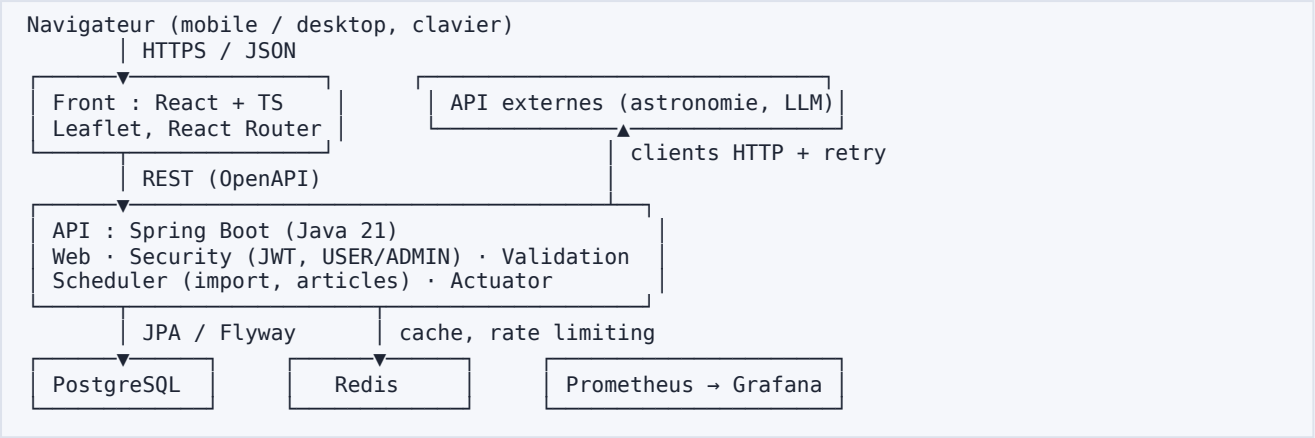
Ce qui la distingue des calendriers astronomiques existants :

- **De la prévision à la sortie collective** : un événement n’est pas qu’une date, c’est un rendez-vous avec participants, forum et notifications de changement.
- **Lieu conseillé justifié** : le site proposé est le meilleur candidat selon la proximité, la fenêtre de visibilité et l’accès, avec une justification affichée. « Optimal » ne signifie pas garantie des conditions réelles, et l’interface le dit.
- **Traçabilité des sources** : chaque fiche affiche sa source et sa date de mise à jour. Une fiche n’est créée que si les données sont suffisantes.
- **IA encadrée** : les articles quotidiens sont générés à partir de faits stockés, puis relus par un humain avant publication. Sans faits fiables, rien d’inventé : repli sur un corpus validé ou absence de publication.
- **Résilience** : si une API tombe, les événements déjà stockés restent consultables.

4. Scénarios d’usage

#	Acteur	Scénario
S1	Visiteur	Ouvre l’accueil, voit les prochains événements près du lieu choisi et l’article du jour, filtre la liste et la carte par date, objet et lieu, sans compte.
S2	Utilisateur	Ouvre la fiche de la pluie des Géminides : horaire local avec fuseau explicite, lieu conseillé et justification, source. Clique « Je participe » et accède au forum de l’événement.
S3	Participant	Publie, modifie ou supprime ses messages dans le forum (pagination, limite de débit). Est notifié si l’horaire ou le statut change.
S4	Utilisateur	Crée une alerte « éclipses, dans un rayon autour de mon lieu préféré » et reçoit une notification in-app à la création d’un événement correspondant.
S5	Utilisateur	Exporte un événement en .ics. Après un changement d’horaire, un nouvel export garde le même UID avec une SEQUENCE incrémentée.
S6	Utilisateur	Exporte la carte en PNG ou l’imprime, avec légende, date et attribution du fond de carte.
S7	Responsable	Relit le brouillon d’article généré dans la nuit, le corrige et le publie, ou le rejette.
S8	Admin	Gère la liste des sites d’observation, modère le forum, gère les rôles.
S9	Tous	L’API externe est indisponible : les fiches stockées restent affichées avec leur date de mise à jour et un bandeau d’information.

5. Architecture et frameworks envisagés



Couche	Technologie	Rôle
Front	React + TypeScript (Vite), Leaflet	Interface responsive et accessible au clavier, carte 2D, export PNG / impression
API	Spring Boot (Java)	API REST, logique métier, sécurité, tâches planifiées, génération .ics
Base de données	PostgreSQL + migrations Flyway	Événements, sites, utilisateurs, participations, messages, alertes, articles
Cache	Redis	Cache des réponses des API externes, limitation de débit du forum
Conteneurs	Docker Compose	Exécution et isolation du front, de l'API, de PostgreSQL, Redis et du monitoring
CI/CD	GitHub Actions	Lint, tests, migrations, build des images, déploiement en staging
Déploiement	Ansible	Configuration reproductible d'un PC ou de VM locales
Supervision	Prometheus + Grafana	Métriques Actuator / Micrometer, tableaux de bord locaux
Infra (facultatif)	Terraform	Uniquement si des VM sont créées chez un fournisseur compatible

Choix structurants. Horaires stockés en UTC et affichés dans le fuseau de l'utilisateur avec mention explicite. Données externes normalisées dans un modèle commun avant stockage, avec source et date de mise à jour. Une seule participation active par couple (utilisateur, événement), garantie par une contrainte d'unicité en base. Tous les écrans gèrent les états chargement, vide, erreur et événement annulé.

Tests visés. JUnit 5 + Testcontainers (API), Vitest + Testing Library (front), quelques parcours de bout en bout. Couverture mesurée par JaCoCo et Vitest, publiée sur Codecov.

6. Protocole d'usage de l'IA

Dans le produit. Le LLM ne reçoit que les événements stockés des 24 dernières heures : il reformule, il n'est pas une source. Tout article est un brouillon tant qu'un responsable ne l'a pas relu et publié. Sans faits fiables, repli sur le corpus « Le savais-tu ? » validé, ou aucune publication. Le prompt, le modèle et la date sont journalisés avec chaque brouillon. Aucune donnée personnelle n'est envoyée au fournisseur.

Dans le développement.

1. Les assistants IA (génération de code, rédaction, revue) sont autorisés comme aides, jamais comme auteurs non relus.
2. Tout code issu d'une IA est compris, testé et relu en Pull Request par un autre membre. Son auteur doit pouvoir l'expliquer à l'oral.
3. Les informations techniques fournies par une IA (API, versions, licences) sont vérifiées dans la documentation officielle.
4. Ce protocole est versionné dans `/doc` et mis à jour à chaque livraison.

7. Dépôts

Dépôt	Contenu	URL
Principal (monorepo)	`/frontend`, `/backend`, `/infra` (Compose, Ansible), `/doc`	https://github.com/celia050/Projet-Orbit-Track

Organisation du dépôt : branche `main` protégée, branches `feature/*`, Pull Requests obligatoires avec CI verte, versions taguées en SemVer (`v0.1.0`, …) et `CHANGELOG.md`. Ce dossier est maintenu en Markdown dans `/doc/dossier-lancement.md`, compilé en PDF à chaque livraison.

README et badges. Le `README.md` à la racine du dépôt affiche les badges de version (tag GitHub), licence (MIT), couverture de tests (Codecov), qualité de code (SonarCloud) et statut de la CI, ainsi que les instructions de lancement (`docker compose up`).